

BIG DATA ANALYTICS & APPLICATIONS

Lecture 03: Distributed Storage & File Systems

Aifee Aadil

What this lecture teaches:

1. Why one disk is too slow, and the trade-offs of distributed systems (CAP).
2. How HDFS stores data: NameNode, DataNodes, blocks, replication, self-healing.
3. How file formats, compression, cloud storage, and Lakehouses make storage fast and cheap.

1. The Storage Bottleneck

From Lectures 1–2: data is huge (Volume), fast (Velocity), and mixed (Variety).

Now the question is: **where and how do we physically store it?**

One disk is too slow

One hard disk reads about **200 MB/sec** (sequential read).

$$1 \text{ PB on 1 disk} \rightarrow 10^{15} \div (200 \times 10^6) = 5 \text{ million sec} \approx \mathbf{58 \text{ days}}$$

↓

$$1 \text{ PB on 1,000 disks in parallel} \rightarrow 5,000 \text{ sec} \approx \mathbf{83 \text{ minutes}}$$

Sequential read = reading data in order, one block after another (fastest way to read a disk).

Simple:

Many disks reading together are far faster than one disk.

Key question: how do we organise and manage **thousands of disks** safely?

Scale-Up vs Scale-Out (storage view)

	Scale-Up (Vertical)	Scale-Out (Horizontal)
How	Faster CPU, RAM, SAN disks on 1 machine	Add cheap servers to a cluster
Cost	Exponential (special hardware)	Linear (normal hardware)
Limit	Hard physical limit	Thousands of nodes
Safety	Expensive hardware backup (RAID)	Software copies (replication)
Big Data fit	Hits a capacity wall	Standard for Hadoop, Spark

RAID = Redundant Array of Independent Disks → several disks inside one machine copying each other, so one disk failure loses no data.

Easy Memory Trick:Scale-Up protects data with **hardware** (RAID)
(replication)Scale-Out protects data with **software**

Schema-on-Write vs Schema-on-Read (recap)

1. Schema-on-Write (RDBMS, ETL)

Define structure → Check data → Store

- Pros: fast queries, clean data.
- Cons: rigid; slows down fast incoming data.

2. Schema-on-Read (Data Lake, ELT)

Store raw → Decide structure at query time

- Pros: accepts any format and speed; cheap storage.
- Cons: slower queries; can become a **Data Swamp**.

Data Swamp = a data lake full of messy, unlabelled raw data that nobody can find or trust.**Example:**

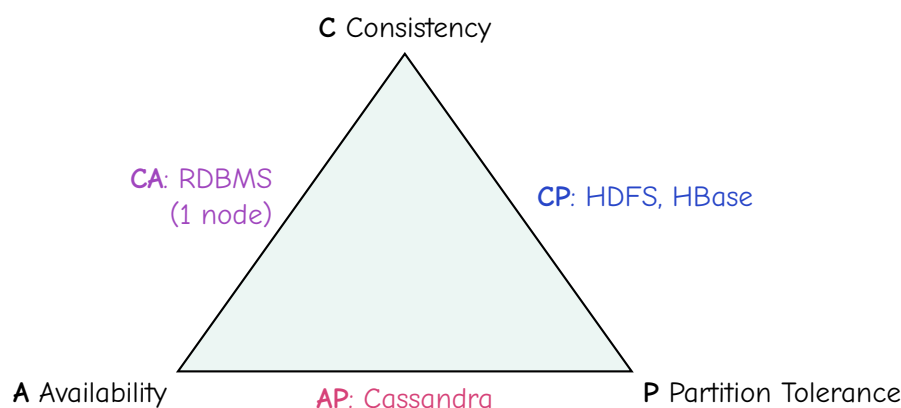
A store room where everything is thrown in without labels → you have it, but can't find it.

2. Distributed System Trade-offs

CAP Theorem

A distributed system can fully guarantee only **2 of these 3**:

- **C — Consistency** → every read gets the **latest** write (or an error).
- **A — Availability** → every working node **always replies** (no error).
- **P — Partition Tolerance** → system keeps working even if the **network breaks** between nodes.

Network partition = some nodes can't talk to others (cable cut, packets lost).

In a big network, **partitions will happen** → P is a must.

So real systems choose:

- **CP** → correct data first; may refuse to answer during a network problem. (HDFS master, HBase)
- **AP** → always answer; data may be a little old. (Cassandra)

Example:

Network breaks between Dhaka and Rangpur servers.

CP: "Sorry, can't confirm the latest balance now."

AP: "Here is the balance" (may be slightly old).

Simple:

When the network breaks, choose: correct (CP) or always answering (AP).

Easy Memory Trick:

C = Correct **A** = Always answers **P** = survives broken network → pick 2

ACID vs BASE (recap)

ACID (RDBMS)	BASE (Big Data)
Atomicity → all or nothing	Basically Available → always works
Consistency → always valid	Soft state → may change by itself
Isolation → transactions don't mix	Eventually consistent → copies match later
Durability → saved forever	
Used in: banking, OLTP	Used in: analytics, clickstreams, social feeds

3. Why Normal File Systems Fail

POSIX = Portable Operating System Interface → the standard rules of normal file systems (Linux, macOS).

1. File locking

POSIX locks a file during writes. Locking across **1,000 servers** → huge network delay.

2. Small blocks

Normal block size = **4–64 KB**. A petabyte file = billions of tiny blocks → too many disk **seeks**. (**Seek** = the disk head moving to the right spot before reading.)

3. Metadata can't scale

Inode = the record that stores a file's details (size, owner, location).
Billions of files across network drives → inode system breaks.

4. No data locality

POSIX keeps storage and compute separate → moving terabytes over the network → network jam.

Simple:

Normal file systems were made for one computer, not thousands.

Move Compute to Data

Old way (bad for Big Data)

Storage server (10 TB) → network → Compute server = network jam

Big Data way (Data Locality)

Code (10 KB) → network → Storage nodes → runs where data already is

Simple:

Send the small code to the big data.

4. Why Big Blocks? (128 MB)

HDFS uses **128 MB** or **256 MB** blocks, not 4 KB. Here's why.

- **Seek time** ≈ 10 ms (time to find the spot).
- **Transfer rate** ≈ 100 MB/sec (reading speed).
- **Goal:** seek time should be only **1%** of the total time.

Seek = 10 ms → must be 1% of time

↓

So transfer time = 100×10 ms = **1 second**

↓

In 1 second at 100 MB/s we read $\approx$ **100 MB**

↓

So blocks of about **128 MB**

Why?

1. Big blocks turn **random** reads (many jumps) into **sequential** reads (one long read) → fast.
2. Fewer blocks → fewer metadata entries in the NameNode's RAM.

Example:

Reading a 1 GB file:

4 KB blocks → ~250,000 seeks. 128 MB blocks → only 8 seeks.

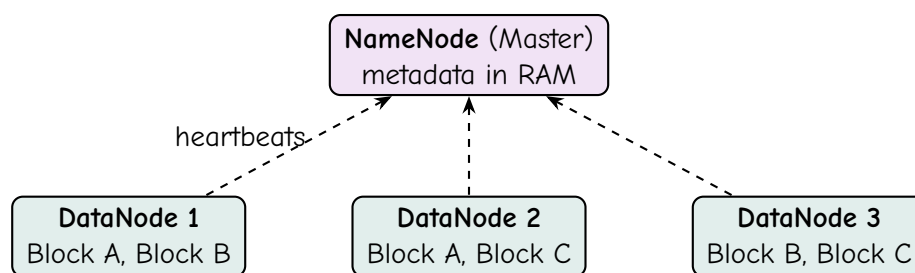
Simple:

Big blocks = less searching, more reading.

5. HDFS Architecture

HDFS = Hadoop Distributed File System → stores huge files across many machines.

1 Master (NameNode) + Many Workers (DataNodes)



Each block is stored on more than one DataNode (copies).

1. NameNode (the Master)

The NameNode **knows where everything is**, but stores no actual data.

- Manages the **namespace** = folder tree, file names, permissions.
- Maps each file → its list of blocks.
- Tracks which DataNodes hold each block.
- Keeps **all metadata in RAM** → very fast lookups.

RAM rule

Every file, block, or folder uses about **150 bytes** of NameNode RAM.

1 million objects → ≈ 150 MB RAM

↓

1 billion small files → ≈ **150 GB RAM!**

Simple:

NameNode = the library catalogue. It knows where every book is.

Takeaway: avoid the **Small Files Problem**.

2. DataNodes (the Workers)

DataNodes **store the actual blocks** on their local disks.

- Store blocks as plain files on the local disk (**ext4/xfs** = normal Linux file systems).
- **Don't know** which file a block belongs to — only the NameNode knows.
- Send a **heartbeat** ("I'm alive") to the NameNode every **3 seconds**.
- Send a **block report** (list of all their blocks) every **10 minutes**.
- Send data **directly to clients** — data does not pass through the NameNode.

Simple:

DataNodes = the bookshelves. They hold the books but don't know the catalogue.

Easy Memory Trick:

NameNode → **Names** and locations (metadata)

DataNode → **Data** (blocks)

3. NameNode High Availability (HA)

Problem: if the only NameNode dies, the whole HDFS stops (**Single Point of Failure**).

Fix: keep a backup NameNode ready.

Active / Standby

- **Active NameNode** → serves all requests.
- **Standby NameNode** → a warm backup that keeps copying the metadata.
- **ZooKeeper + ZKFC** (ZooKeeper Failover Controller) → notices if Active dies and switches to Standby automatically (**failover**).

Quorum Journal Manager (QJM)

- Active NameNode writes every change to an **odd number** of **JournalNodes** ($2N + 1$, e.g. 3 or 5).
- A write counts only if a **quorum** (majority) saved it: 2 of 3, or 3 of 5.
- Standby reads from the JournalNodes to stay in sync.

Quorum = majority vote.

Why an odd number?

With 3 JournalNodes, 1 can fail and a majority (2) still exists. An odd number avoids ties.

Active writes change → 2 of 3 JournalNodes save it → Standby reads it

Simple:

A backup NameNode stays in sync and takes over automatically.

4. FSImage & EditLog

Writing the full metadata to disk on every change would be too slow. So the NameNode uses 2 files:

- **FSImage** → a full **snapshot** (photo) of all metadata at one moment.
- **EditLog** → a list of every change since then (create, delete, rename). It is a **write-ahead log** (change is written to the log first).

Checkpointing

Old FSImage + EditLog → merge → new FSImage

The Secondary/Standby NameNode downloads both, merges them, and sends the fresh FSImage back to the Active NameNode.

Example:

FSImage = last month's bank statement. EditLog = transactions since then.
Merge them → today's statement.

Why?

Without checkpoints the EditLog grows huge and a restart takes very long.

Simple:

FSImage = snapshot, EditLog = changes. Merge them regularly.

5. HDFS Commands

1. Create a folder

```
hdfs dfs -mkdir -p /user/cis312/data
```

2. Upload a file (split into 128 MB blocks)

```
hdfs dfs -put local_dataset.csv /user/cis312/data/
```

3. List files (human-readable sizes)

```
hdfs dfs -ls -h /user/cis312/data/
```

4. Check blocks, locations, health

```
hdfs fsck /user/cis312/data/local_dataset.csv -files -blocks -locations
```

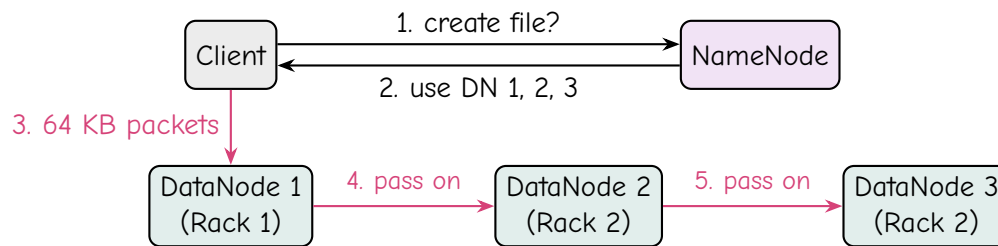
5. Show the first 10 lines

```
hdfs dfs -cat /user/cis312/data/local_dataset.csv | head -n 10
```

- `-mkdir -p` → make folder (and parent folders).
- `-put` → copy from your computer into HDFS.
- `-ls -h` → list files, sizes like "2.1 G".
- `fsck` → File System Check → shows blocks, where they are, and if any are missing/corrupt.
- `-cat ... | head` → print file content (first 10 lines).

6. Read & Write Pipelines

1. Writing a file



1. Client asks the NameNode to create a file.
2. NameNode replies with a list of **3 DataNodes**.
3. Client sends data in **64 KB packets** to DataNode 1.
4. DataNode 1 passes each packet to DataNode 2.
5. DataNode 2 passes it to DataNode 3.

This chain is called a **pipeline**.

Key point: data goes **directly** from client to DataNodes. The NameNode only gives directions — it is not a **proxy** (middleman) for data.

Simple:

Ask NameNode where to write → send data down a chain of 3 DataNodes.

2. Reading a file

1. Client calls `open()` on NameNode with the file path
- ↓
2. NameNode finds which DataNodes hold each block
- ↓
3. NameNode sorts them by **closeness** to the client (network topology)
- ↓
4. Client calls `read()` and connects **directly** to the closest DataNode
- ↓
5. If that node fails or data is corrupt → client switches to the next copy

- **Network topology** → the map of how machines are connected (same node → same rack → other rack).
- **FSDataInputStream** → the Java object the client uses to read the data.
- **Checksum failure** → the data is damaged (see Checksums below).

Simple:

Ask NameNode where the blocks are → read from the nearest DataNode → switch copy if it fails.

Easy Memory Trick:

NameNode gives the **address**, DataNodes give the **data**.

7. Fault Tolerance

1. 3x Replication

HDFS stores **3 copies** of every block by default.

Advantages	Disadvantages
No data loss even if 1 or 2 nodes crash	300% storage: 100 TB data needs 300 TB disk
Programs can run on any copy in parallel	Lots of network traffic while uploading
Auto-repair when copies drop below 3	

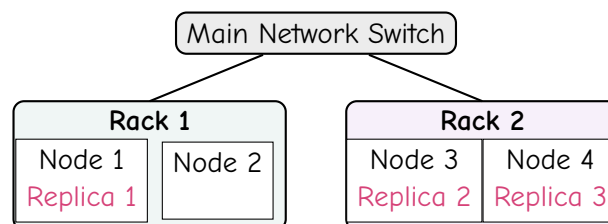
Simple:

3 copies = safe, but 3× the disk space.

2. Rack Awareness

Rack = a cabinet of servers sharing one power supply and one network switch.

If a rack's switch or power fails, **all its nodes go down together**.



- **Replica 1** → on the local node (local rack).
- **Replica 2** → on a node in a **different rack** → survives Rack 1 failure.
- **Replica 3** → on another node in that **same remote rack** → less cross-rack traffic.

Why not put all 3 on different racks?

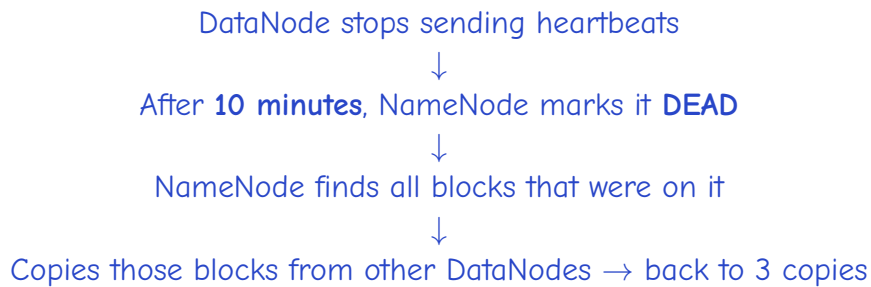
Sending data between racks is slower and uses the main switch. Two racks already protect against a whole-rack failure.

Simple:

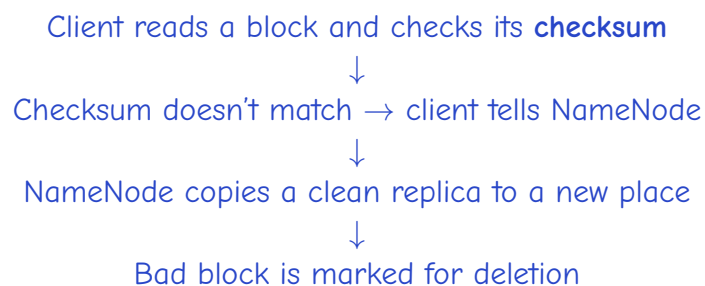
Copies on 2 racks: safe from rack failure, but not too much traffic.

3. Self-Healing

Case 1: DataNode crashes



Case 2: Corrupted block



Simple:

HDFS finds and fixes lost or damaged blocks by itself.

4. Checksums & Block Scanner

Checksum = a small number calculated from the data. If the data changes even by 1 bit, the checksum changes.

CRC-32 checksums

- HDFS calculates a **CRC-32** checksum for every **512 bytes**. (CRC = Cyclic Redundancy Check.)
- Stored in hidden **.crc** files.
- Checked during writing, reading, and in the background.

DataBlockScanner

- A background program (**daemon**) on every DataNode.
- Checks all its blocks regularly (about every **3 weeks**).
- Finds **bit rot** (data slowly getting damaged on old disks) **before** anyone reads it.

Example:

Checksum is like counting the pages of a book before and after lending it. If the count differs, a page is missing.

Easy Memory Trick:Heartbeat → is the **node** alive?Checksum → is the **data** healthy?

8. File Formats

Row vs Column layout

ID	Name	Salary
1	Alice	5000
2	Bob	6000

1. Row-oriented

`[1, Alice, 5000] | [2, Bob, 6000]`

Whole rows stored together.

2. Column-oriented

`[1, 2] | [Alice, Bob] | [5000, 6000]`

Each column stored together.

Easy Memory Trick:Row → good for **writing** recordsColumn → good for **analysing** columns

Row formats: CSV, JSON, Avro

CSV / Text

- Easy to read, works everywhere.
- Problems: not compressed, no schema inside, slow text parsing.

JSON

- Supports nested data.
- Problem: very wordy — key names repeated in every row.

Example:`{"name": "Aadil", "age": 22}` → the words `name` and `age` are stored again in every record.

Apache Avro

- Compact **binary** row format; its schema is written in JSON.
- Great for streaming (Kafka → landing area of the lake).
- Best **schema evolution** (add/remove columns smoothly).

Column formats: Parquet, ORC

Apache Parquet

- The standard columnar format (Spark, Trino, Arrow, DuckDB).
- Data split into **row groups** (128 MB – 1 GB).
- Each column compressed separately with:
 - **RLE** (Run-Length Encoding) → “BD, BD, BD, BD” → “BD × 4”.
 - **Dictionary encoding** → replace repeated words with small numbers (Dhaka = 1, Rangpur = 2) and store the numbers.

Apache ORC (Optimized Row Columnar)

- Built for Hive, Presto, Hadoop.
- Keeps **stripe statistics** (min, max, sum, count of each column part).
- Supports ACID in Hive.

Simple:

Row formats for writing/streaming (Avro). Column formats for analytics (Parquet, ORC).

Pushdown: reading less data

1. Projection Pushdown (skip columns)

Query: `SELECT name FROM users;`

- Row storage → reads every column (ID, Name, Salary, Address, Email...).
- Parquet → reads **only** the Name column.
- Saves up to **90%** of disk reading.

Projection = choosing which columns you want.

2. Predicate Pushdown (skip rows)

Query: `... WHERE age > 65;`

- Parquet stores **min and max age** for each block.
- Block says min = 18, max = 45 → no one is over 65 → **skip the whole block**.

Predicate = the WHERE condition.

Easy Memory Trick:

Projection → fewer columns

Predicate → fewer rows/blocks

Compression

Codec	Compression	CPU use	Splittable in HDFS?
Gzip	High (~70%)	High	No (unless indexed)
Bzip2	Very high	Very high	Yes
Snappy	Medium (~40%)	Very low	No (but yes inside Parquet)
ZSTD	High	Balanced	Yes

Codec = a compression method (coder–decoder).

Why splittable matters

If a compressed file is **not splittable**, Spark/MapReduce can't give different parts to different workers.

Not splittable → 1 worker reads the whole file alone → slow

Splittable → many workers read parts together → fast

Example:

A 10 GB Gzip file → only 1 worker can unzip it, start to end.

Easy Memory Trick:

Snappy = **fast**, light Gzip/Bzip2 = **small**, heavy ZSTD = **balanced**

9. Cloud Storage & Lakehouse

HDFS vs Cloud Object Storage

Object storage = store files as “objects” in the cloud (**S3**, **GCS**, **ADLS** = Azure Data Lake Storage).

	On-Premises HDFS	Cloud Object Storage
Compute + storage	Coupled (same nodes do both)	Decoupled (scale separately)
Cost	High fixed cost (buy servers)	Pay per GB per month
Speed	Very fast local disk	More network delay, huge throughput
Scaling	Limited by number of nodes	Practically unlimited (elastic)
Maintenance	Need own admin team	Managed by cloud provider

- **Coupled** → if you need more storage, you must also buy more CPUs (same machines).
- **Decoupled** → add storage without adding compute, and the other way round.

Example:

HDFS = owning your own warehouse building.

Cloud storage = renting space and paying only for what you use.

Simple:

HDFS = fast but fixed and self-managed. Cloud = flexible, pay-as-you-go, managed for you.

From Warehouse to Lakehouse

Data Warehouse (1990s) → structured, ACID, expensive, schema-on-write



Data Lake (2010s) → raw files on HDFS/S3, cheap, schema-on-read, **no ACID**



Data Lakehouse (now) → cheap object storage + open table formats

Open table formats: Delta Lake, Apache Iceberg, Apache Hudi.

They add a smart metadata layer on top of Parquet files.

What a Lakehouse adds

- **ACID transactions** on cloud files (no half-written data).
- **Time travel** → query older versions of the data (“show the table as it was last Monday”).
- **Schema enforcement and evolution** without rewriting tables.

Simple:

Lakehouse = cheap like a lake + safe like a warehouse.

Easy Memory Trick:

Warehouse → **safe**, costly

Lake → **cheap**, messy

Lakehouse → **both**

10. Case Study: The 10 TB Query

Data: 10 TB log file with **1,000 columns**.

Query: average transaction amount where status = 'FAILED'.

Option A: Raw CSV

- Must read **all 10 TB**.
- Must parse text line by line.
- Time: about **14 hours**.

Option B: Parquet + Snappy

- Reads only **2 columns** (amount, status) → **projection pushdown**.

- Skips **80%** of row groups using min/max → **predicate pushdown**.
- Time: about **45 seconds!**

2 of 1,000 columns + skip 80% blocks + compressed → 14 hours becomes 45 seconds

Simple:

The right file format can make a query over 1000× faster.

11. Quick Revision

Remember

- 1 PB: 1 disk $\approx$ 58 days; 1,000 disks $\approx$ 83 minutes.
- Scale-Out uses software replication instead of RAID.
- CAP: pick 2 of C, A, P. P is a must → CP (HDFS, HBase) or AP (Cassandra).
- POSIX fails: locking, tiny blocks, inode limits, no locality.
- 128 MB blocks: seek = 1% of transfer time; less NameNode RAM.
- NameNode = metadata in RAM ($\sim$ 150 bytes/object). DataNodes = blocks.
- Heartbeat every 3 s; block report every 10 min; dead after 10 min.
- HA: Active + Standby, ZooKeeper/ZKFC, QJM quorum (2 of 3).
- FSImage = snapshot, EditLog = changes, checkpoint = merge.
- Write: client → 3 DataNodes pipeline (64 KB packets). Read: nearest DataNode.
- 3 copies on 2 racks. CRC-32 per 512 bytes. Scanner every 3 weeks.
- Parquet/ORC + projection & predicate pushdown → read much less.
- Splittable compression lets many workers read in parallel.
- Lakehouse = object storage + Delta/Iceberg/Hudi (ACID, time travel).

12. Exam Questions

Q1. Why does HDFS use 128 MB blocks instead of 4 KB?

Ans: Disk seek $\approx$ 10 ms and transfer $\approx$ 100 MB/s. To keep seek time about 1% of the total, each read should transfer for $\approx$ 1 second $\approx$ 100 MB, so blocks are about 128 MB. Big blocks turn random reads into sequential reads, and fewer blocks means less metadata in the NameNode's RAM.

Q2. What happens when a DataNode crashes?

Ans: It stops sending heartbeats. After 10 minutes the NameNode marks it dead, finds all blocks that were on it, and copies them from the remaining replicas to healthy DataNodes until each block has 3 copies again. No human is needed.

Q3. Explain the CAP theorem. Why do Big Data systems choose CP or AP?

Ans: A distributed system can guarantee only 2 of Consistency, Availability, and Partition

tolerance. In large networks partitions always happen, so P is required. Systems choose CP (correct data, may refuse requests; e.g. HDFS, HBase) or AP (always answers, data may be briefly old; e.g. Cassandra).

Q4. Why is Parquet + Snappy so much faster than CSV in the 10 TB case?

Ans: Projection pushdown reads only the 2 needed columns out of 1,000; predicate pushdown skips 80% of row groups using min/max statistics; the data is compressed and binary, so no text parsing. Read time drops from about 14 hours to about 45 seconds.

Practice Questions

1. How long does reading 1 PB take on 1 disk vs 1,000 disks?
2. Compare Scale-Up and Scale-Out for storage. What is RAID?
3. What is a Data Swamp?
4. Why do POSIX file systems fail at Big Data scale?
5. Explain the roles of NameNode and DataNode.
6. Calculate NameNode RAM for 1 billion small files.
7. How does NameNode High Availability work (Active/Standby, ZooKeeper, QJM)?
8. What are FSImage and EditLog? What is checkpointing?
9. Explain the HDFS write pipeline and read process.
10. Explain rack awareness and the 3 replica placement rules.
11. How do checksums and the DataBlockScanner protect data?
12. Compare CSV, JSON, Avro, Parquet, and ORC.
13. Explain projection pushdown and predicate pushdown.
14. Why must compression be splittable?
15. Compare HDFS and cloud object storage.
16. What is a Lakehouse? What does it add to a Data Lake?

Next lecture: Lecture 04 — Distributed Resource Management with **YARN** and **MapReduce** basics.